

AiScientist: A Privacy-First Multi-Agent Research Console for Single-Cell and Ocular Genomics

System implementation, capability evaluation, and model hosting requirements

UCI RUIC20 Lab · Version 0.2.0 “DAG” · 2026-09-05

Contents

1	Abstract	1
2	Introduction	3
2.1	The problem	3
2.2	The approach	3
2.3	Naming	3
3	Methods	4
3.1	System architecture	4
3.2	The agent research lab	5
3.3	Analysis tool implementation	6
3.4	Model evaluation design	6
3.5	Estimating self-hosting capacity and GPU device counts	6
3.6	Design of the mechanism validations	7
4	Results	9
4.1	Deployed capabilities	9
4.2	Research scenarios covered	9
4.3	Example analysis output	11
4.4	Model evaluation results	15
4.5	GPU hosting requirements	17
4.6	Mechanism validation results	19
5	Discussion	20
	Supplementary material	21
S1	Operating guide	21
S2	Tool catalog	23
S3	Technology stack	25
S4	Glossary	26
S5	Key environment variables	26
S6	Path conventions	27
S7	Repository layout	27
S8	Supplementary figures	28
S9	Sources	30

1 Abstract

Single-cell and spatial transcriptomic analysis is a long chain of judgement calls: QC thresholds, clustering resolution, the choice of differential-expression test, the choice of enrichment library, marker interpretation, and the alignment of conclusions

with the literature. That chain is currently walked by hand across several tools, and the parameter choices are rarely recorded.

We built **AiScientist**, a privacy-first multi-agent bioinformatics research console deployed on the UCI campus network (ai-scientist.eye.som.uci.edu). A researcher asks a scientific question in plain language in the browser and points at a dataset; the system connects to UCI HPC3 over SSH, serves an open-weights LLM on a Slurm GPU via vLLM, drives a PI → Scientist → Critic role loop that calls real analysis tools, and returns a citable PDF/DOCX manuscript with figures and tables plus a downloadable results bundle. Raw data never leaves the campus network, and all heavy compute runs on the cluster as Singularity-contained Slurm jobs.

The system comprises 39,916 lines of Python across 8 packages, 1,611 offline tests, 30+ named tools, 15 rewritable atomic skills and 7 preset pipelines, covering nine classes of research scenario including condition-contrast differential expression, cell-type annotation, variant interpretation and phenotype-driven differential diagnosis. We illustrate the output with one real run (15,307 cells × 33,696 genes, producing a 12-page manuscript, 26 figures and 80+ tables).

We further built a model-substitutability evaluation framework that compares ten model configurations across planning, execution and writing **while holding the scaffolding constant and swapping only the model**, and from it derive both a role-split model recommendation and an estimate of the GPU device counts required for self-hosting across five commercially available Blackwell devices (RTX PRO 4500/5000/6000, B200, B300). Four key mechanisms were independently validated against known answers or the real served model.

2 Introduction

2.1 The problem

Every step of a single-cell analysis involves a defensible but non-unique choice. Whether the mitochondrial QC threshold is 5% or 10% depends on whether the sample is cells or nuclei; the clustering resolution determines every downstream cell-type label; whether a condition contrast belongs at the cell level or aggregated to samples depends on the number of biological replicates; and a sequencing-depth imbalance between arms will disguise a technical effect as biology. These judgements are scattered across different tools and different people's habits, and are seldom recorded in full.

2.2 The approach

AiScientist implements that chain as a **supervised** agent team, under three design principles.

Privacy first. The model runs on a UCI HPC3 GPU; weights and inference never leave the campus network. Raw datasets exist only server-side and on HPC3, and the gateway keeps no copy. `DataBoundaryGuard` inspects the prompt before any model call and blocks raw data rows and secrets. Outbound literature queries are sanitized to a bare filename and the research question; deep literature retrieval uses the session's local model endpoint and a local embedding model, so corpus chunks stay on campus.

Heavy compute stays on the cluster. GPU inference, the code sandbox, scanpy analysis, report rendering, scGPT annotation and visual review all run as Singularity-contained Slurm jobs on HPC3. File transfers use a dedicated transfer host, in line with RCIC's rules on login-node use.

Results are checkable. Tools report their real status, returning `dependency_missing` rather than pretending to have run. A **deterministic guard** — code, not a prompt — constrains what the reviewing role may accept. Report assembly involves no model, and figures and tables are embedded exactly as the tools wrote them.

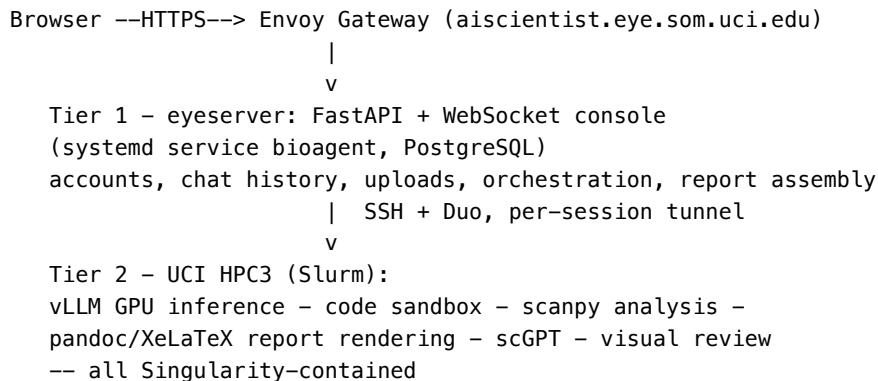
2.3 Naming

The product is **AiScientist** (formerly BioAgent) in all user-facing text. The code namespace stays `bioagent`: the package `src/bioagent/`, the `BIOAGENT_*` environment variables, the production data directory `/data/BioAgent`, and the systemd unit and SSH service account `bioagent`. `core.config.apply_brand_env_aliases()` mirrors `BIOAGENT_*` and `AISCIENTIST_*` when `.env` is loaded, so either prefix resolves.

3 Methods

3.1 System architecture

The system is two tiers, decoupled by per-session SSH tunnels and Slurm job submission. The web tier holds no GPU and no persistent copy of any dataset.



The code is organised as follows.

Package	Responsibility	Files	Lines
agents/	The research lab (PI/Scientist/Critic), DAG planner, agent memory, tool registry, code sandbox, provenance	20	10,701
gateway/	Web console: SSH+Duo, Slurm vLLM serve and tunnel, accounts, chat history, uploads, report assembly, HPC3 offload	33	15,383
tools/	Real analysis (scanpy/gseapy), literature (Europe PMC + PaperQA2), report (pandoc), scGPT, visual review, schematics, variants, phenotype	30	11,990
integrations/	The DataBoundaryGuard safety layer	5	761
lab/	Research-lab support	3	468
providers/	OpenAI-compatible LLM client (with fallback)	2	266
hpc/	Slurm boundaries	2	226
core/	Shared config, brand env aliases	2	113
Total		97	39,916

The frontend is 5,244 lines of vanilla JavaScript (no framework, served from disk per request), with Cytoscape for the DAG view and Mermaid for inline diagrams.

Data boundaries. Data crosses three boundaries during a run:

Stage	Where the data is	Boundary
Upload	Browser → gateway (chunked) → the user's HPC3 area	The gateway keeps no copy
Analysis	HPC3 compute node, read and written inside the container	The raw matrix never enters a prompt

Stage	Where the data is	Boundary
Model call	The prompt passes DataBoundaryGuard, then the local vLLM	First: blocks raw rows and secrets
Literature	Only keyword search leaves campus	Second: a bare filename and the question
Deep literature	PaperQA2 runs entirely on HPC3	Third: the corpus stays on campus
Results	HPC3 → gateway → browser	Derived results only

The uniform tool contract: take a dataset **path**, return **derived metrics, figure paths or gene lists** — never the raw expression matrix.

Deployment. Production is the systemd unit `bioagent.service` on eyeserver, bound to the node IP on `:8800`, behind a selector-less Kubernetes Service and Envoy Gateway. Because each session’s SSH tunnel and connection state live in one process’s memory, the deployment uses `replicas: 1` with a `Recreate` strategy; the production database is PostgreSQL 17 (SQLite is for development and CI only); run artifacts are written to a persistent volume; the session-cookie signing key has no default, and the admin is seeded from a bcrypt hash.

3.2 The agent research lab

The live agent system is a role loop in `src/bioagent/agents/`. **The three roles share one set of model weights**; what separates them is scaffolding — prompts, execution contracts and deterministic guards.

Role	Input	Output
PI	Question + dataset profile + tool roster	An ordered agenda
Scientist	One step’s brief	A sequence of tool calls and a final answer
Critic	Step description + tool artifacts	<code>accept / revise</code> , a score, a specific critique
Coordinator	The ready set	The next task to run (DAG mode)
Specialists	A task description	Claim by expertise fit and execute (DAG mode)

The **dataset profile** is computed deterministically before the run and carries cells per arm, cells per cell type per arm, median sequencing depth with a depth-imbalance marker, replication status, and single-cell versus single-nucleus protocol inference. With the tool roster it forms the PI’s planning input.

The per-step contract. At most 8 tool calls per step; each result must be read before continuing; the step ends by returning its final answer through `finish`. Tool results are truncated per entry before entering context, bounding a step’s prompt size.

The deterministic accept guard. The reviewing role is a model, so a guard outside the model sits alongside it: if a step’s tool call errored or returned nothing, the Critic may not accept it. The guard is code rather than a prompt, so the model cannot route around it, making the chain “tool fails → review passes → report narrates an analysis that never happened” structurally impossible. Plan review carries four further written rules: is every specified operation computable; is every promised output produced by some step; does a step’s description match what its named tool actually does; and does a literature step ask rather than assert.

Report assembly involves no model. On convergence: the accepted findings and the figures and tables the tools wrote are collected; `_synthesize` writes the narrative prose (the only model-authored writing step); `tools/report.py` renders the PDF via `pandoc` → `XeLaTeX` and the DOCX via `pandoc`; figures and tables are embedded exactly as produced, with no model redrawing a data figure; the reference section is generated deterministically from the accepted citations; and process detail and any degradations go to a separate technical report, leaving the manuscript clean. Four anti-fabrication layers apply in order: correct data, closed-set grounding, number-format and wording rules, and caption-truth checking. A report can be regenerated without re-running the analysis, and a single analysis step can be re-run on its own.

Plan revision patches rather than redrafts. The user gives feedback in natural language in plan mode; the model returns only `{"step": n, "new_text": ...}` and code applies the change, so steps the user never mentioned cannot change because they are never regenerated. Only a request that a one-step edit cannot express falls back to a full redraft.

DAG planning, multi-agent and agent memory (added in 0.2.0, feature-flagged and additive): a structure pass turns the agenda into a `LabPlan` with explicit dependencies and a ready-set scheduler starts nodes as prerequisites complete, each node

receiving a scoped brief; a Coordinator picks the next ready task and specialists claim by expertise fit; branches with disjoint mutable footprints run in parallel; and each specialist keeps a private, disk-backed memory (`episodes.jsonl` plus a distilled `lessons.md`) recalled into its brief and updated by reflection after a run — in-context learning on frozen weights.

3.3 Analysis tool implementation

The tool roster is assembled dynamically by `agents/registry.py`: when scanpy is available the lightweight smoke tools give way to the real analysis line; when the corresponding executors are injected the tools submit HPC3 jobs while keeping an in-process fallback; and with no HPC session bound the filesystem tools are absent rather than present-but-broken. The roster spans five groups — the single-cell analysis line, the variant and phenotype lines, the literature line, code and files, and meta tools — totalling 30+ named tools; the full catalog is in Supplementary S2.

Key implementation points: `run_clustering` can select the Leiden resolution by bootstrap resampling scored with the adjusted Rand index, keeping the finest resolution that still clears the stability threshold; `run_de` fixes its output columns at `group, gene, log2fc, pval, pval_adj, score` and supports condition contrasts stratified by cell type; `run_pseudobulk_de` aggregates to the sample level and applies pydeseq2’s negative-binomial Wald test; `run_depth_matched_de` quantile-matches the deeper arm onto the shallower arm’s per-cell UMI distribution, re-runs the same test and Spearman-correlates per direction; enrichment and GSEA are offline against local `.gmt` gene sets; and in `map_phenotype_to_hpo` the model extracts phrases and detects negation while the ontology owns the term IDs — the model can only select a candidate number, so a non-existent HPO term is impossible.

3.4 Model evaluation design

The core of the design is to **hold the scaffolding constant and swap only the model**: every arm runs the real production code paths (calling the ResearchLab methods directly rather than re-implementing them), on a real dataset (`Ddx41_DEG.h5ad`, 15,307 cells), calling real tools. Differences between arms are therefore attributable to the model, and behaviour shared by all arms to the scaffolding.

Stage	What runs	Held constant	Scoring
A · Plan	<code>ResearchLab._pi_plan</code>	System prompt, protocol guidance, dataset profile, tool roster	A 14-check deterministic rubric plus two blind judges
B · Execute	<code>_scientist</code> → <code>ResearchHarness</code> → <code>_critic</code> , per step	One fixed 7-step plan; the workspace seeded with earlier steps’ outputs; the same persona	Tool-call correctness, argument match, call counts, how the step ends, review acceptance, plus a blind judge
C · Write	<code>_synthesize</code> → <code>_build_report</code> → <code>_review_report</code>	The same accepted findings, figures and tables	Number format, wording consistency, figure embedding, factuality, plus a blind judge

The blind judges are `claude-opus-5` and `gemini-3.1-pro-preview`. The harness is `experiments/plan_vs_exec_ab/run_ab.py`, run in six stages (`prep / A / B / C / judge / report`). The nine arms are listed in the Results.

Conditions. OpenRouter serves Qwen3.6 as fp8 while production is AWQ 4-bit; tools ran locally with no Slurm queue, so wall-clock is not comparable; stage B’s fixed plan was written by the production model and is identical for every arm; stage A used 2–3 repetitions per arm, stage B 6–14 trials, and stage C 2 per prompt variant, so results give magnitude and ordering rather than precise rates. Trials that could not complete in round 1 because of an API quota limit are excluded from every statistic.

3.5 Estimating self-hosting capacity and GPU device counts

To answer “how many of which cards are needed to host a given candidate model”, we use the following procedure.

Step 1 — weight size. NVFP4 quantization stores 4 bits per weight plus one FP8 block scale per 16 values, about 4.5 bits per parameter, i.e.

$$W \approx 0.56 \text{ GB} \times P_{[\text{billions of parameters}]}$$

This is an upper bound. Our own two NVFP4 checkpoints are smaller (126–127 GB for a 304B model, about 0.42 GB per billion parameters) because the non-expert layers are a small share and some layers stay FP8. Where a checkpoint size was measured, the table uses the measured value.

Step 2 — aggregate VRAM needed to serve. Beyond the weights, serving must also hold activations, CUDA-graph capture, MoE routing buffers and a working KV pool. We calibrate this factor on our own two measured points:

- Qwen3.6-35B-A3B (20 GB of weights) serves on a **single** 96 GB card while leaving a 58.8 GiB KV pool;
- DeepSeek-V4-Flash-0731 (127 GB of weights) requires **TP=4** on 96 GB cards; TP=2 runs out of memory.

Since TP=2 (192 GB aggregate) is insufficient while TP=4 (384 GB aggregate) works, the required aggregate lies between 1.5× and 3.0× the weight size. We take

$$V_{\text{required}} \approx 1.8 \times W$$

as the working estimate, which is consistent with both measured points (Qwen3.6: 36 GB ≤ 96 GB, one card; DSV4-Flash: 229 GB, three 96 GB cards rounded up to TP=4). Two further measured deployments agree with it: MiniMax-M2.7 at FP8 (220 GB) and MiniMax-M3 at NVFP4 (228 GB) both load at TP=4 on the same 384 GB node, i.e. at 1.75× and 1.68× their weight size, so 1.8× is a safe planning figure and is occasionally conservative by one device tier.

Step 3 — device count.

$$N = \lceil V_{\text{required}} / C_{\text{per device}} \rceil$$

rounded up to a legal tensor-parallel size. vLLM and SGLang require the tensor-parallel size to divide the attention-head count, which in practice means 1, 2, 4, 8 or 16.

Step 4 — interconnect. The RTX PRO series connects over PCIe and offers no NVLink, so at equal device counts its tensor-parallel scaling efficiency is lower than an HGX B200/B300 system with NVLink. The device-count table answers whether a model fits, not whether throughput is equal.

The commercially available Blackwell devices considered, with vendor-published per-device memory (the 96 GB tier is the device we measured on):

Device	Class	Memory per device	Interconnect
RTX PRO 4500 Blackwell	Workstation	32 GB GDDR7	PCIe
RTX PRO 5000 Blackwell	Workstation	48 GB GDDR7	PCIe
RTX PRO 6000 Blackwell (Workstation / Max-Q / Server)	Workstation / server	96 GB GDDR7	PCIe
NVIDIA B200	Data centre (HGX)	180 GB HBM3e	NVLink 5
NVIDIA B300 (Blackwell Ultra)	Data centre (HGX)	288 GB HBM3e	NVLink 5

Sources differ on the B200, quoting either 180 GB or 192 GB per device; we use the more conservative 180 GB.

3.6 Design of the mechanism validations

Beyond unit tests, four key mechanisms were validated against a **known answer** or the **real served model**. The designs are below; results are in the Results section.

Plan-revision fidelity. The same model, the same plan, 4 edit requests × 5 repetitions on the real vLLM-served Qwen3.6, comparing whole-plan redraft against a code-applied patch. Metrics: did the target step change, was the intent satisfied, **how many other steps changed**, and did the step count move. The scorer was first self-checked against synthetic perfect / collateral / dropped-step / no-op plans.

Known-answer validation of the depth-matching tool. Synthetic data with a known answer: 800 cells and 300 genes per cell type, with the MUT arm sequenced about 2× deeper per cell than WT. The RealBio type contains 30 genes whose expression fraction really is 3× higher (correct verdict: preserved); the DepthOnly type has identical fractions in both arms (correct verdict: not preserved). The validation was run twice: first on 2026-08-20, and again on 2026-09-02 after a fix to

gene selection, to where the robustness floor is applied, and to how the two directions are treated. The second run is the current result and is the one reported below.

Evidence distribution in multi-agent meetings. The same meeting, the same six findings, the same three experts and the same synthesis prompt, with the single variable being whether each expert sees all the evidence or its own slice. The six findings deliberately pair a strong per-cell signal with a design fact that limits its interpretation. n=1 per arm, as a mechanism illustration.

An auditable protocol format. An operon-style `PROTOCOL.md` was written for the variant-annotation pipeline (a summary table with per-step collapsible detail: what, why, the agent-chosen parameters, the real command, a verify checklist), with commands taken from the real source. The old and new document bodies were A/B tested against the same planner prompt, scored deterministically out of 11, plus an LLM judge and a readability rating.

4 Results

4.1 Deployed capabilities

Capability	Built	Enabled in production
Web console (accounts, SSH/Duo, Slurm vLLM, tunnels, GPU isolation, mid-run auto-recovery)	Yes	Yes
The linear PI→Scientist→Critic lab	Yes	Yes
The real scanpy/gseapy analysis line	Yes	Yes
The code sandbox (CodeAct)	Yes	Yes
Deterministic pandoc PDF/DOCX reports, regenerable without re-run	Yes	Yes
The literature line (Europe PMC + PaperQA2)	Yes	Yes
Server-side chat history and resumable upload	Yes	Yes
The phenotype line (HPO mapping + LIRICAL + differential)	Yes	Yes
Per-agent evolving memory	Yes	Yes
Bring-your-own LLM API key	Yes	Yes
The fast Chat path and inline Mermaid	Yes	Yes
Patch-based plan revision	Yes	Yes
The depth-matched validation tool	Yes	Yes
The DAG planner	Yes	Optional
Real multi-agent with expert claiming	Yes	Optional
Safe concurrency	Yes	Optional
The offline VEP variant line	Yes	Optional
scGPT per-cell annotation	Yes	Optional
Vision-model review of the rendered report	Yes	Optional
The LangGraph execution shell	Yes	Optional
Hypothesis-driven exploration / multi-cycle loops / skill induction	Yes	Optional
The model evaluation framework	Yes	—

Scale: 39,916 lines of Python (97 files, 8 packages), 1,611 offline tests (no cluster, .env or network), 675 commits since 2026-06-08, 5,244 lines of frontend, 15 atomic skills and 7 preset pipelines.

4.2 Research scenarios covered

Scenario 1 — condition-contrast single-cell differential expression (preset pipeline `differential_expression`). KO-vs-WT, disease-vs-control or treated-vs-untreated snRNA-seq / scRNA-seq datasets. The complete chain:

Step	Tool	Implementation
Dataset profile	<code>inspect_dataset</code>	Deterministically computes cells per arm, cells per cell type per arm, median depth, replication status, single-cell vs single-nucleus inference
Quality control	<code>run_scanpy_qc</code>	Per-cell metrics, cell and gene filtering, normalisation, log1p, HVG selection; writes QC figures and a checkpoint

Step	Tool	Implementation
Doublents	<code>run_doublent_detection</code>	Scrublet, after QC and before clustering
Batch integration	<code>run_integration</code>	Corrects donor and batch effects when the object holds several samples
Clustering	<code>run_clustering</code>	PCA → neighbours → Leiden → UMAP; optional stability-selected resolution
Differential expression	<code>run_de</code>	Wilcoxon rank-sum; a reference level gives a condition contrast, a cell-type column runs it within each type
Aggregated test	<code>run_pseudobulk_de</code>	One profile per sample, pydeseq2 negative-binomial Wald
Depth correction	<code>run_depth_matched_de</code>	Quantile-matches UMI depth, re-runs the same test, Spearman per direction
Composition	<code>run_composition</code>	Cell-type proportions per sample and their shift between conditions
Pathways	<code>run_enrichment</code> / <code>run_gsea_prerank</code>	Offline ORA and preranked GSEA, per group and per direction
Literature	<code>literature_search</code> / <code>deep_literature</code>	Queries built from the accepted findings; cited conclusions
Manuscript	Report assembly	pandoc → XeLaTeX PDF and DOCX, figures embedded as produced

Scenario 2 — cell-type annotation for unlabelled datasets (`celltype_annotation`, `scgpt_annotation`). Stability-selected resolution turns “resolution” from an unexamined assumption into a defensible choice; `run_marker_annotation` assigns types from a marker panel with lineage-specific discriminators; `scgpt_annotate` gives every cell a label and confidence from a separate short-lived GPU batch job; a dataset that already carries labels is reused rather than recomputed.

Scenario 3 — variant interpretation for inherited eye disease and rare disease (`variant_annotation`). The production path is offline VEP with a local cache on HPC3, containerised as a Slurm job, handling WGS-scale VCFs; annotation covers consequence, affected gene, predicted impact and ClinVar significance; pathogenicity plugins are CADD, REVEL, AlphaMissense and OpenSpliceAI; the genome assembly is auto-detected from the VCF header; the toolchain container carries bcftools 1.21, samtools, tabix, bedtools and pysam/cyvcf2.

Scenario 4 — phenotype-driven differential diagnosis (`phenotype_variant_diagnosis`). `map_phenotype_to_hpo` converts free text in any language into validated HPO terms; `run_lirical` calls LIRICAL 2.4.1 and ranks candidate diseases by calibrated post-test probability; `diagnose_disease` combines that probability with PaperQA2 literature evidence into an adjudicated differential.

Scenario 5 — gene signature scoring and perturbation analysis (`gene_signature_scoring`, `perturbation_analysis`), including escape-cell filtering, perturbation-versus-control contrasts, e-distance effect sizes and stratified comparisons.

Scenario 6 — literature grounding and citable output. Keyword queries are built at run time from the accepted findings rather than templated from the user’s question; deep retrieval runs entirely on HPC3 with a local model and local embeddings; the reference section is generated deterministically from accepted citations.

Scenario 7 — fast question answering. The Chat path answers in seconds with a hand-picked short list of tools (literature search, HPO mapping, deep literature); the bar for inclusion is in-process execution, a seconds-scale return, and no run workspace or Slurm job. That list is built from the same tool definitions as the research path, so the two cannot drift.

Scenario 8 — compliant compute and data placement. Uploads stream straight into the user’s own HPC3 area with no gateway copy; transfers use a dedicated transfer host while compute and tunnels stay on the login node; GPU jobs are per-user isolated; per-run process files are collected in a shared scratch directory swept on a 3-day cycle.

Scenario 9 — bring your own model endpoint. A user may add their own OpenAI-compatible endpoint and key in place of the session model, with keys stored encrypted server-side; an HPC3 account remains required because the analysis jobs run on the cluster.

4.3 Example analysis output

The following comes from the real run c135ae589d96 (2026-09-02) on the lab’s own DDX41 retina single-nucleus RNA-seq data.

Item	Value
Data	Ddx41_DEG1.h5ad, 15,307 cells × 33,696 genes
Design	sampleid with two levels (DDX41 6,260 / WT 9,047)
Labels	The file carries majorclass (11) and celltype (87); reused, not recomputed
Output	A 12-page manuscript PDF, DOCX, a technical report, 26 figures, 80+ tables
Rendering	pandoc → XeLaTeX

The manuscript is structured as abstract, dataset, what was run, study question, results (quality control, clustering, differential expression, pathway enrichment, depth-matched validation and pseudobulk, composition and literature grounding), discussion, limitations, conclusion, methods and references.

The “what was run” section is the core of its reproducibility: each analysis is attributed to the named tool that performed it, every parameter is listed with an explanation, and any non-default value is marked as chosen for that run. In this run the mitochondrial QC threshold was set to 5% rather than the tissue default of 10%, with the reason recorded: the data was judged to be single-**nucleus** (a nuclear-fraction column is present and median mitochondrial content is only 1.5%), and a nucleus should carry almost no mitochondrial signal.

Single-cell transcriptomic profiling of DDX41-depleted retinal nuclei reveals depth-driven transcriptional shifts

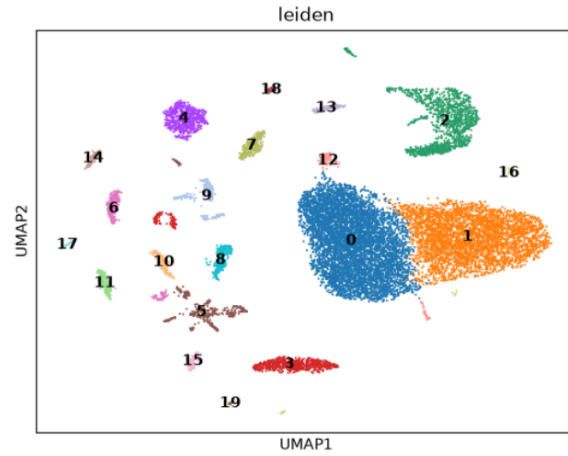
Contents

Abstract	1
The dataset	2
What was run	3
Study question	5
Results	5
Quality control	5
Cell clustering	6
Differential expression	7
Pathway enrichment	9
Depth-matched validation and pseudobulk	9
Cell-type composition and literature grounding	10
Discussion	10
Limitations	11
Conclusion	11
Methods	11
References	12

Abstract

DDX41 is a conserved splicing factor implicated in photoreceptor maintenance and retinal dystrophy, yet its precise transcriptional impact on non-photoreceptor retinal subtypes remains uncharacterized at single-cell resolution. We profiled a single-donor, snRNA-seq dataset comparing DDX41-depleted retinal nuclei to wild-type (WT), processing 15,307 high-quality nuclei through quality control, stability-selected Leiden clustering, stratified Wilcoxon contrasts, depth-matched validation, pseudobulk aggregation, and over-representation/preranked gene set enrichment analysis. The dataset resolved 20 transcriptional clusters aligning with 11 annotated retinal major classes, with initial stratified contrasts yielding thousands of genes exhibiting large effect sizes across five cell types. However, depth-matched differential expression revealed that zero of eight cell-type direction rankings survived UMI equalization (Spearman $\rho < 0.5$), and three rankings completely inverted, while pseudobulk aggregation returned zero statistically supported genes. Pathway shifts were dominated by translational and metabolic processes, but these signals are conditional on descriptive rankings confounded by a 1.1–1.7 $\times$ sequencing depth imbalance. Consequently, all observed transcriptional and pathway-level shifts are framed as descriptive effect-size hypotheses rather than validated regulatory outcomes. Independent validation using donor-replicated snRNA-seq, matched sequencing depth, and orthogonal molecular assays is required to distinguish technical

Figure 1: The sample manuscript’s first page: title, contents and abstract



mixed clusters.

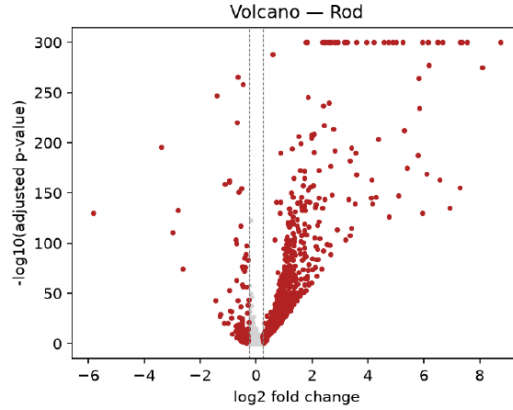
Differential expression

The core take-home message is that initial stratified contrasts identify genes with large descriptive effect sizes across five major classes, but these rankings are highly sensitive to sequencing depth imbalance and do not represent validated differential expression. Stratified Wilcoxon contrasts with `groupby='sampleid'`, `reference='WT'`, and `stratify_by='majorclass'` were applied to five major classes meeting the minimum 30-cells-per-arm threshold: AC, BC, Cone, MG, and Rod. Six classes were excluded from analysis due to falling below the stability threshold. The genes with the largest positive and negative log2 fold-changes in each class are summarized below:

majorclass	total_sig	up_regulated	down_regulated	total_genes_tested	min_abs_log2fc	max_abs_log2fc
AC	515	374	140	8371	0.30	4.91
BC	1910	1463	284	7309	0.22	3.83
Cone	383	329	53	6840	0.38	5.99
MG	7182	5028	508	9720	0.48	26.32
Rod	4350	3279	107	4649	0.51	8.74

Volcano plots displaying genes past the descriptive threshold for each class are shown below:

Figure 2: Page 7: the UMAP of 20 Leiden clusters and the per-cell-type differential-expression summary



Pathway enrichment

The core take-home message is that coordinated pathway shifts in translation and metabolism are detected by ORA and preranked GSEA, but these signals are conditional on descriptive, depth-confounded rankings and should not be interpreted as validated biological pathways. Over-representation analysis tested up- and down-regulated gene halves ($|\log_2 \text{fold-change}| \geq 0.25$) against GO, Reactome, and Hallmark libraries. Preranked GSEA walks on Wilcoxon-statistic ranked lists identified coordinated shifts. In BC, genes with the largest positive $\log_2$ fold-changes enriched for translation-related terms, while genes with the largest negative $\log_2$ fold-changes enriched for phototransduction (NES -2.7). In Cones, down-regulated rankings showed strong depletion of carbohydrate catabolic and visual perception pathways (NES -3.1). MG and Rod rankings similarly highlighted translational elongation and phototransduction cascades. These enrichment p-values are descriptive overlap statistics computed on a possibly depth-confounded ranking; they report the strength of gene-set overlap but do not confirm that the underlying expression changes are biological.

Depth-matched validation and pseudobulk

The core take-home message is that sequencing depth artifacts fully explain the initial differential expression and enrichment signals, as depth equalization collapses all rankings and pseudobulk aggregation yields zero supported genes. The DDX41 libraries sampled 1.1–1.7 $\times$ deeper UMIs per cell than WT. After down-sampling to match WT distributions within each major class, Wilcoxon contrasts were re-run. Zero of eight cell-type $\times$ direction rankings survived depth matching (Spearman $\rho < 0.5$), and three rankings completely inverted, indicating pure detection artifacts. Pseudobulk aggregation (summed raw counts $\rightarrow \log_2 \text{CPM} \rightarrow \text{Welch } t\text{-test}$) yielded zero significant genes across all tested classes, con-

9

Figure 3: Page 9: the Rod volcano, pathway enrichment and depth-matched validation

All figures are drawn deterministically by scanpy and matplotlib and embedded as the files they are, with no model involved in drawing. Further examples are in Supplementary S8.



Figure 4: The deterministic workflow schematic from the schematic tool, recording the step chain this run executed

Depth-matched validation in practice. Effect sizes from the initial stratified contrasts across five cell types:

majorclass	Significant	Up	Down	Genes tested	Mean abs log2FC	Max abs log2FC
AC	515	374	140	8,371	0.30	4.91
BC	1,910	1,463	284	7,309	0.22	3.83
Cone	383	329	53	6,840	0.38	5.99
MG	7,182	5,028	508	9,720	0.48	26.32
Rod	4,350	3,279	107	4,649	0.51	8.74

The depth-matched review found median per-cell UMIs differing $1.1\text{--}1.7\times$ between arms (DDX41 3078, WT 1916); none of the eight cell-type $\times$ direction rankings held after UMI equalisation (Spearman $p < 0.5$) and three inverted; and pseudobulk aggregation returned no statistically supported genes in any tested class. The system accordingly framed the results as descriptive hypotheses and recorded in Limitations that donor replication, matched depth and orthogonal molecular assays would be required. This is precisely the purpose of the capability: to separate the depth effect from biology before a conclusion reaches the manuscript.

One qualifier on how to read that verdict. This run’s report was written at 18:36 on 2026-09-02, and the direction fix described in the Methods and in §4.6 landed later the same evening, so the run used the earlier tool, which applied one p threshold to both directions and treated both as testable. Under the current tool the four DOWN rankings would be reported as `against_depth_untestable` rather than counted as failures, so the summary “none of eight rankings held” would today read as none of the four testable UP rankings holding, with the four DOWN rankings neither validated nor refuted. The conclusion for the UP direction is unchanged.

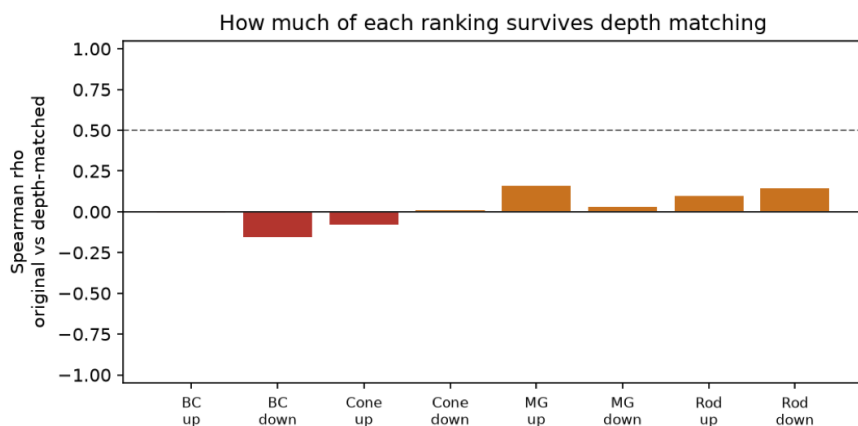


Figure 5: The depth-matched correlation output of `run_depth_matched_de`

4.4 Model evaluation results

4.4.1 Arms

Arm	Model	Note
qwen36-35b	qwen/qwen3.6-35b-a3b	The production Scientist loop (thinking off)
qwen36-35b-think	same	The production PI / Critic / writer (thinking on)
qwen35-122b	qwen/qwen3.5-122b-a10b	A larger open-weights model
deepseek-v4-pro	deepseek/deepseek-v4-pro	A strong open-weights model
sonnet-5	anthropic/claude-sonnet-5	Frontier reference
gpt-5.4	openai/gpt-5.4	Frontier reference
minimax-m2.7	minimax/minimax-m2.7 (229B-A10B)	Candidate for our own node
minimax-m3	minimax/minimax-m3 (428B-A23B, multimodal)	Candidate for our own node
deepseek-v4-flash	deepseek/deepseek-v4-flash-0731 (304B, MIT)	Candidate for our own node
laguna-s-2.1	poolside/laguna-s-2.1	Added after the first two rounds; stages A and B only

4.4.2 Stage A — plan quality

Arm	n	Steps	Rubric	Halluc. tools	Depth-aware	Replication-aware	Judge: sound	Specific	Dataset fid.	Seconds
qwen36-35b	3	8.33	0.91	0	0%	100%	6.00	6.83	7.50	24.9
qwen36-35b-think	3	7.67	0.98	0	67%	100%	7.17	7.67	8.17	285.0
qwen35-122b	3	9.00	0.94	0	33%	100%	6.33	7.33	8.00	13.2
deepseek-v4-pro	3	11.00	0.93	0	100%	100%	8.50	9.17	9.50	44.8
sonnet-5	3	11.33	0.98	0	100%	100%	9.50	9.50	10.00	79.8
gpt-5.4	3	15.67	0.83	0	100%	100%	9.33	9.33	9.67	29.6
minimax-m2.7	2	9.00	1.00	0	100%	100%	8.50	8.50	8.75	53.9
minimax-m3	2	11.00	1.00	0	100%	100%	9.25	9.50	9.75	252.4
deepseek-v4-flash	2	10.00	1.00	0	100%	100%	9.50	9.50	9.75	162.6
deepseek-v4-flash-low	2	8.00	0.97	0	100%	100%	9.50	9.50	9.75	41.8
laguna-s-2.1	3	6.33	0.89	0	100%	100%	9.33	9.33	9.83	156.3

The fundamentals — stratified DE, composition analysis, enrichment only after DE, the replication caveat, no hallucinated tools — are met by **every** arm. Separation appears on the item that requires judgement, namely whether the plan acts on the depth imbalance flagged in the dataset profile, and it tracks model capability monotonically.

4.4.3 Stage B — executing the same plan

Arm	Trials	Named tool called	OK	Arg match	Sandbox calls/step	Recon/step	Finished w/ answer	Critic accept
qwen36-35b	14	86%	86%	1.0	3.36	3.79	29%	71%
qwen36-35b-think	14	93%	93%	1.0	3.00	3.79	36%	93%
qwen35-122b	12	92%	92%	1.0	1.42	4.92	17%	92%
deepseek-v4-pro	6	67%	67%	1.0	2.33	5.00	50%	67%
minimax-m2.7	14	93%	93%	1.0	4.00	2.50	50%	71%
minimax-m3	14	93%	93%	1.0	2.93	5.71	43%	79%
deepseek-v4-flash	14	86%	86%	1.0	4.43	4.86	21%	79%
deepseek-v4-flash-low	14	86%	86%	0.9	4.36	3.79	21%	71%
laguna-s-2.1	14	57%	57%	1.0	2.00	8.79	14%	29%

Tool-call correctness is high for every arm but one: 86–93% of steps call the tool the step names, and the argument match is 1.0 wherever a tool is called (0.9 for DeepSeek-V4-Flash at low effort). The differences lie in convergence behaviour after the call, where MiniMax-M2.7 makes the fewest reconnaissance calls (2.50 per step) and most often finishes with an answer

(50%). Code that ran clean on the first attempt: MiniMax-M3 95%, DeepSeek-V4-Flash 90%, MiniMax-M2.7 86%, Qwen 79–82%.

The one arm that departs from the pattern is `laguna-s-2.1`, added after the first two rounds: it calls the step’s named tool in only 57% of steps and makes 8.79 reconnaissance calls per step — more than three times MiniMax-M2.7 — finishing with an answer in 14% of steps and drawing a Critic acceptance of 29%, the lowest of any arm. Its planning is strong (stage A judge soundness 9.33, dataset fidelity 9.83 on the shortest agendas of any arm at 6.33 steps), so it is a clear instance of the plan/execute split this design was built to separate. It was not run for stage C.

4.4.4 Stage C — writing the same findings

Arm	n	Chars	“significant” claims	Depth mentioned	Judge: raises artefact	Replication consistent	Invents	Quality
qwen36-35b	2	21,501	4.0	1	25%	100%	25%	4.25
qwen36-35b-think	2	22,309	0.0	3	75%	75%	75%	5.50
qwen35-122b	2	24,581	4.0	1	0%	100%	50%	3.25
deepseek-v4-pro	2	28,469	4.0	2.5	75%	100%	100%	4.75
sonnet-5	2	23,818	2.0	9	100%	100%	0%	8.50
minimax-m2.7	2	41,739	6.5	9.5	75%	100%	100%	4.50
minimax-m3	2	27,967	3.0	4.5	100%	100%	50%	7.50
deepseek-v4-flash-low	2	26,003	2.0	9.5	100%	100%	0%	8.00

(`laguna-s-2.1` and `gpt-5.4` were not run for stage C.)

The writing-stage prompt rules (number format, replication wording, descriptive DE, caption truth) take effect for every arm and remove mechanical errors such as scientific notation and invented captions; what remains is judgement, which tracks model capability. A further actionable result: a reasoning model needs an output budget sized for its thinking — DeepSeek-V4-Flash at low effort with a 6× budget reaches 8.0, second only to Sonnet 5.

4.5 GPU hosting requirements

Applying the method in Methods §5, the GPU device counts required for each candidate model on five commercially available Blackwell devices are as follows. Entries are “count (tensor-parallel size)”, already rounded up to a legal tensor-parallel size.

Model	Params (total / active)	NVFP4 weights	Aggregate VRAM needed	RTX PRO 4500 32 GB	RTX PRO 5000 48 GB	RTX PRO 6000 96 GB	B200 180 GB	B300 288 GB
Qwen3.6-35B-A3B (production)	35B / 3B	20 GB	36 GB	2 (TP2)	1	1 [measured]	1	1
Qwen3.5-122B-A10B	122B / 10B	68 GB	123 GB	4 (TP4)	3 → 4 (TP4)	2 (TP2)	1	1
MiniMax-M2.7	229B / 10B	128 GB (FP8: 220 GB [measured])	231 GB	8 (TP8)	5 → 8 (TP8)	4 (TP4) [measured]	2 (TP2)	1
MiniMax-M3	428B / 23B	228 GB [measured]	384 GB [measured]	12 → 16	8 (TP8)	4 (TP4) [measured, loads]	3 → 4 (TP4)	2 (TP2)

Model	Params (total / active)	NVFP4 weights	Aggregate VRAM needed	RTX PRO 4500 32 GB	RTX PRO 5000 48 GB	RTX PRO 6000 96 GB	B200 180 GB	B300 288 GB
DeepSeek-V4-Flash-0731	304B / 13B	127 GB [measured]	229 GB	8 (TP8)	5 → 8 (TP8)	3 → 4 (TP4) [measured]	2 (TP2)	1
DeepSeek-V4-Pro	not stated	—	—	—	—	—	—	—
Claude Sonnet 5 · GPT-5.4	closed	—	—	not self-hostable				

Notes:

- The two cells marked [measured] are configurations we verified on real hardware: Qwen3.6-35B-A3B serves on a single RTX PRO 6000 96 GB card while leaving a 58.8 GiB KV pool; the DeepSeek-V4-Flash-0731 NVFP4 weights serve at TP=4 across four 96 GB cards (SGLang 0.5.18 with `--moe-runner-backend marlin`, 512 K context, ~78 tok/s), while TP=2 runs out of memory.
- “3 → 4” means the arithmetic count is 3, but the tensor-parallel size must be a power of two, so four cards are deployed.
- MiniMax-M2.7’s measured deployment uses the **official FP8 weights** (220 GB on disk, about 55 GB per card at TP=4) rather than an NVFP4 checkpoint; either basis lands on four cards at the 96 GB tier.
- MiniMax-M3’s NVFP4 checkpoint measures **228 GB** (88 of 88 shards verified) and loads at TP=4 on the 4×96 GB node, i.e. at 1.68× its weight size — below the 1.8× planning figure, so for this model the estimate is conservative by one device tier. Memory is not M3’s constraint; see the serving-stack note below.
- The table excludes multi-user concurrency. Serving several sessions simultaneously requires scaling the KV pool accordingly. Qwen3.6’s hybrid attention makes its KV unusually cheap (about 20 KiB per token, below), whereas a conventional dense-GQA model typically costs several times more per token.

Our present ceiling. The `free-gpu32` partition provides four concurrent GPUs for the whole `ruic20_lab` account, on nodes carrying four RTX PRO 6000 96 GB cards each (32 CPU, 257 GB RAM), i.e. **384 GB aggregate**. From the table, Qwen3.6-35B, Qwen3.5-122B, MiniMax-M2.7 and DeepSeek-V4-Flash can all be hosted within that ceiling, and MiniMax-M3’s NVFP4 checkpoint also loads at TP=4 within it — M3’s obstacle is the serving stack rather than memory (below). The partition is free but preempt-cancel with a 3-day maximum; the paid `gpu` partition is non-preemptible with a 14-day maximum, and a 96 GB card bills at twice the service-unit rate.

Measured KV and context. The production model Qwen3.6-35B-A3B uses hybrid attention: `layer_types` is linear attention except every 4th layer, so only 10 of its 40 layers hold a KV cache, at about 20 KiB per token. A full 262 K context therefore costs about 5 GiB of KV, and 1 GB of KV buys roughly 51 K tokens; `max_position_embeddings` is natively 262,144. Measured on the real cluster on 2026-08-02:

Card	Partition	KV cache	Tokens	Concurrency @262K
A100 80 GB PCIe (sm_80)	gpu (paid)	47.8 GiB	2,466,442	9.41×
RTX PRO 6000 Blackwell 96 GB (sm_120)	free-gpu32 (free)	58.8 GiB	3,035,461	11.58×

The NVFP4 serving stack. DeepSeek-V4-Flash’s NVFP4 weights serve on `sm_120` with SGLang 0.5.18 and `--moe-runner-backend marlin` at TP=4 (512 K context, ~78 tok/s, tool calls correct); stock vLLM 0.28.0 also runs unpatched at TP=4 with a 131 K context.

The MiniMax serving stack (measured). MiniMax-M2.7’s official FP8 weights (220 GB on disk, about 55 GB per card at TP=4) serve on `sm_120` under **vLLM 0.22.1** with `--tool-call-parser minimax_m2`, TP=4 and a **196,608-token context** as reported by `/v1/models`, with chat and tool calls verified. vLLM 0.27.1 and the current nightly hang indefinitely in the warm-up forward pass — all four GPUs at 100% for over four hours — so the version must be pinned to 0.22.1. MiniMax-M3’s NVFP4 checkpoint (228 GB, 88 of 88 shards verified) loads and serves at TP=4 on the same 4×96 GB node but produces deterministically corrupted output on every stable vLLM: M3’s NVFP4 support ships in `vllm-project/vllm #46380` and

is not yet in a stable release, and the stable path fails silently rather than raising. As a control, Llama-3.1-8B-FP4 is correct on the same node and image at both TP=1 and TP=4, so the hardware, the FP4 dequantisation and tensor parallelism are all sound.

4.6 Mechanism validation results

Plan-revision fidelity.

Implementation	Clean	Any collateral change	Mean collateral	Length changed	Intent satisfied
Whole-plan redraft (n=19)	0%	100%	3.89	89%	53%
Code-applied patch (n=19)	100%	0%	0.00	0%	100%
Production (patch, n=20)	100%	0%	0.00	0%	100%

The patch framing removed collateral changes and also lifted intent satisfaction from 53% to 100%. The result additionally shows that a larger model buys nothing for plan editing, and the production implementation was settled accordingly.

Known-answer validation of the depth-matching tool (2026-09-02, after the selection, robustness and direction fix; HPC3 analysis container).

Cell type	Direction	Spearman ρ	Verdict
DepthOnly	up	0.15	weak — correctly not preserved
DepthOnly	down	-0.05	against_depth_untestable
RealBio	up	0.84	preserved
RealBio	down	0.32	against_depth_untestable

At gene level, RealBio up keeps **27 of 30** of the genuinely changed genes and **0 of 30** of the background; the earlier rule kept 30 of 30 but also passed 3 of 30 background, so specificity is now perfect at the cost of three true genes. DepthOnly up passes 19 of 54 pure-background genes, and there the guard is the ranking verdict weak rather than the gene count.

Only one of the two directions is testable, and the tool now says so. Depth inflates detection in the deeper arm, so it can only manufacture apparent UP-regulation there. The DOWN direction runs against that gradient, so a low ρ is not evidence of an artefact — but down-sampling the deeper arm also pushes every gene toward looking more down, so the check cannot confirm those genes either: measured here, a surviving-effect rule passed 92% of pure background in RealBio down at a 0.5 floor and 73% at 0.8. The tool therefore reports **against_depth_untestable** with **no robustness count at all** for that direction, rather than a number that would certify noise. Genes in the down direction are neither validated nor refuted by this check.

The robustness floor sits on the Wilcoxon z , and 0.8 was swept rather than chosen. Floors of 0.5, 0.6, 0.8 and 1.0 keep 30, 30, 27 and 7 of the 30 real genes with 0 of 30 background throughout, while the pure-depth control passes 50%, 43%, 35% and 28%.

Evidence distribution in multi-agent meetings. With shared evidence the discussion settled on methodology in general, citing scVI, MAST and DESeq2 — none of which were in the evidence. With a split, it anchored on this study's evidence, each side citing the table it had actually read, producing an informative positional disagreement; the synthesis output moved with it, from generic rules to specific judgements about this dataset.

An auditable protocol format. The new format is substantially more auditable at no cost to planning quality, for roughly 36% more prompt tokens. The design rule it established: a readable document must be a projection of the code that actually runs, generated from source with a CI staleness check, never a hand-forked copy.

5 Discussion

Model selection. The system already carries role splitting (BIOAGENT_LAB_LLM_*), so different roles can use different models. Across the three stages:

Role	Recommended model	Basis
PI and writer	DeepSeek-V4-Flash-0731	Plan 9.5 (joint best), writing 8.0 (second), MIT-licensed, lowest cost; must be run at low reasoning effort with an output budget of at least 32 k
Scientist loop	MiniMax-M2.7	Best execution behaviour: fewest reconnaissance calls, highest rate of finishing with an answer; interleaved thinking is built for tool use; and it is the one candidate whose serving is already verified on our own node (FP8, TP=4, vLLM 0.22.1, 196,608-token context)

The hosting path. Both models fit the four RTX PRO 6000 cards currently available (384 GB aggregate), each at TP=4 — and for MiniMax-M2.7 this is measured rather than estimated: its official FP8 weights serve at TP=4 under vLLM 0.22.1 with a 196,608-token context and verified tool calls. Serving both roles simultaneously would require eight cards, or data-centre devices with larger per-device capacity — two B200 each, or one B300 each. MiniMax-M3, the strongest all-rounder on the evaluation numbers, also loads at TP=4 within the same 384 GB, so its obstacle is not memory but the serving stack: NVFP4 support for M3 is not yet in a stable vLLM release, and the stable path returns corrupted output silently. Note that the RTX PRO series connects over PCIe without NVLink, so at equal device counts its tensor-parallel scaling is less efficient than an HGX system; the device-count table answers whether a model fits, not whether throughput is equal.

Methodology. Running each arm through the real production code paths rather than a re-implementation is the key design choice: it makes the inference “differences are the model, commonalities are the scaffolding” valid, and it lets an evaluation conclusion translate directly into a configuration change. Likewise, validating `run_depth_matched_de` on synthetic data with a known answer is a step unit tests cannot replace — a unit test can show that code runs as designed, not that the design is scientifically correct.

Scope. The evaluation was run on one dataset and one research question with 2–3 repetitions, so its conclusions give magnitude and ordering. The GPU device-count table is based on vendor-published per-device capacities and a factor calibrated on two measured points; it is intended for capacity planning, not performance prediction.

Supplementary material

S1 Operating guide

Prerequisites

Requirement	Notes
An AiScientist account	Admin-created; if self-registration is enabled it accepts @uci.edu only and requires an emailed code
Network	UCI campus network or VPN
An HPC3 account	Required — even with your own model key, analysis jobs run under your own HPC3 account
Browser	Any modern browser with WebSocket support

Signing in and connecting to HPC3

Go to <https://aiscientist.eye.som.uci.edu/> and enter your username and password. The database stores only a bcrypt hash of the app password; HPC3 credentials are never stored.

Connect from the “CONNECT TO HPC3” panel on the Research page, by one of three methods:

- **Password and Duo:** enter your UCInetID and password and choose a Duo method (push, phone call, passcode); optionally tick Remember me to set up a reusable SSH key on HPC3; confirm you are on the campus network and connect.
- **Saved SSH key:** pick the stored key from the dropdown and supply its passphrase if it has one. Private keys live server-side, mode 0600 and per-user.
- **Your own model endpoint:** the LLM endpoint defaults to the cluster GPU; you may add your own OpenAI-compatible endpoint and key.

Once connected the system starts a per-user Slurm GPU job to serve the model and warms it on connect. Mock mode offers a cluster-free demonstration.

The screenshot shows the 'AiScientist Console' interface. At the top, it says 'SSH → UCI HPC3 · GPU · vLLM / Qwen3'. The top right shows 'Disconnected' and 'BioAdmin · admin'. The left sidebar has icons for Research, Datasets, Runs, System, Admin, and Account. The main area is divided into three columns. The first column, 'CONNECT TO HPC3', has fields for HPC host (hpc3.rcic.uci.edu), UCInetID (BioAdmin), Auth method (Password + Duo), Password (masked), Duo approval (Push — approve on your phone), a 'Remember me' checkbox, a passphrase field, and an LLM endpoint dropdown (Cluster GPU (vLLM at UCI)). The second column, 'New chat', has instructions to connect to a GPU-backed Qwen3 on HPC3, a dropdown for 'Research (full pipeline)', 'Auto (PI decides)', and 'Plan first (PI proposes; refine in chat)', a 'Bypass' checkbox, a 'Data' dropdown, and a 'Send' button. The third column, 'RESULTS', has instructions on how to run a pipeline and browse results.

Figure 6: The Research page. The left column shows the six provisioning stages (SSH login, vLLM detect/image, GPU allocation, vLLM serve and tunnel, model ready, live) above the server-side chat history; run controls are at the lower right.

Uploading a dataset and starting a run

Uploads are chunked and resumable and stream straight into /dfs3b/ruic20_lab/<UCInetID>/uploads/. A single run can bind several data files.

Control	Options	Meaning
Execution path	Research (full pipeline)	The full research pipeline, producing a manuscript
	Chat (fast answer)	Fast question answering, seconds
Team shape	Auto (PI decides)	The PI chooses single or multi-agent (default)
	Single agent	Force single-agent
	Virtual Lab	Force multi-agent
Human gate	Plan first	The PI proposes a plan, refined in chat before execution (default)
	Bypass	Skip the human gate and run autonomously

Advanced lets you search and force a preset pipeline; by default the PI chooses.

Plan mode

With Plan first ticked, the PI produces the agenda and pauses. Read it through, then give feedback in chat in natural language — no manual text editing; the system applies it by the patch mechanism. Once confirmed, execution begins and progress streams back per role, token by token.

Reading and collecting results

The results panel offers the rendered report, every figure, every table and the full results bundle. Two follow-up actions: “Ask the PI to revise the report” rewrites the report without re-running anything; “Ask the PI to re-run an analysis step” re-runs a named step.

The screenshot shows the AiScientist Console interface. On the left is a sidebar with icons for Research, Datasets, Runs, System, Admin, and Account. The main area is divided into two panels. The left panel, titled 'CONNECT TO HPC3', contains fields for HPC host (hpc3.rcic.uci.edu), UCInetID (BioAdmin), Auth method (Password + Duo), Password (masked), Duo approval (Push — approve on your phone), a checkbox for 'Remember me', a passphrase field, and an LLM endpoint dropdown (Cluster GPU (vLLM at UCI)). The right panel, titled 'finish analyse this dataset and generate t', displays a text report about pseudobulk aggregation and testable hypotheses. Below the report are dropdowns for 'Research (full pipeline)' and 'Auto (PI decides)', a checkbox for 'Plan first (PI proposes; refine in chat)', and a 'Bypass' option. At the bottom, there's a text input field with the placeholder 'Ask AiScientist to plan or interpret an ocular bioinformatics analysis...' and a 'Send' button.

Figure 7: The results panel: bundle download, regenerate report, re-run a step, a file browser, and the run’s 11 report files and 27 figures.

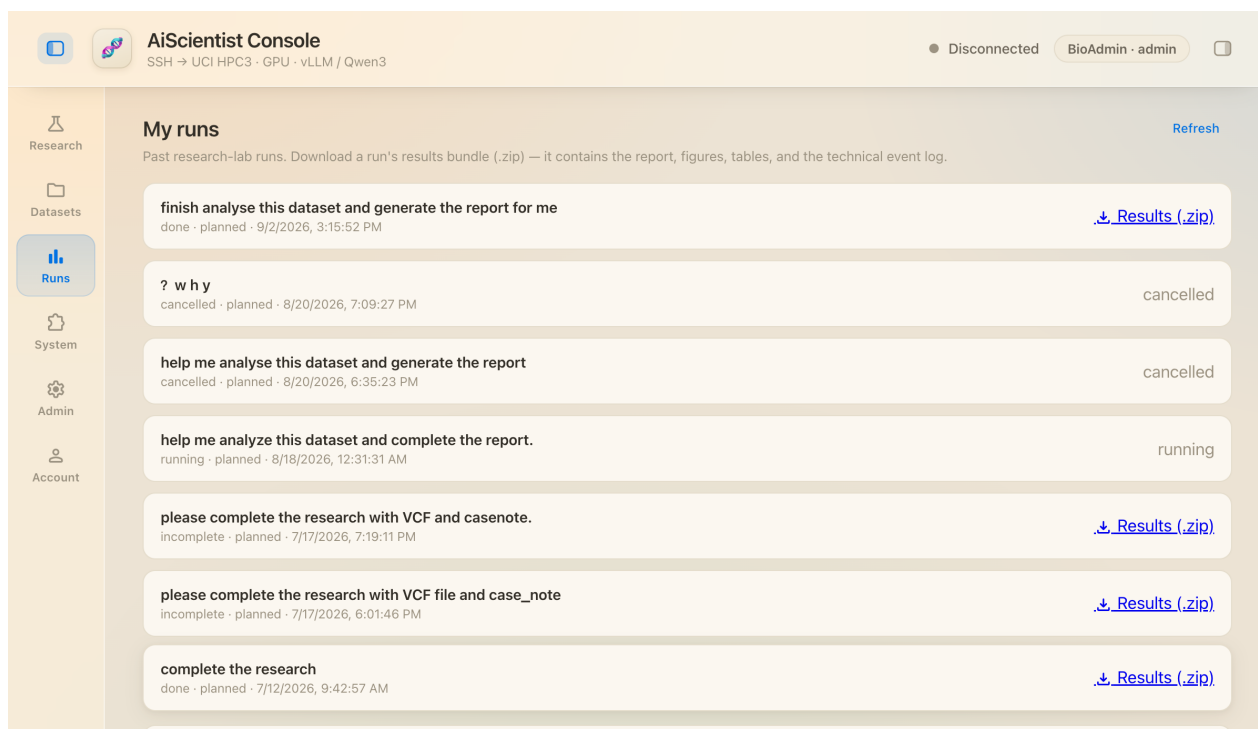


Figure 8: The Runs page: every past run with its status; completed runs offer the results bundle.

Server-side artifact paths:

```
runs/console/<UCInetID>/<run_id>/
  figures/      figures (PNG)
  tables/      tables (CSV)
  report/      report.md/.pdf/.docx – technical_report.* – plan.md
  process/     run_state.json and other process records
  data/        checkpoints
```

Administration and local development

The bioagent-admin command line offers create-admin, reset-password, list-users and hash-password. The Admin page offers account management and the System page shows the tool roster and runtime environment.

```
./deploy.sh           # create the venv and install (idempotent)
./start.sh            # bind 127.0.0.1:8800
python3 -m pytest     # 1,611 offline cases; no cluster, .env or network
```

S2 Tool catalog

The System page generates this roster live from the code: each tool with its name, full description, dependencies and whether it is available on this server; tools that read private data are badged.

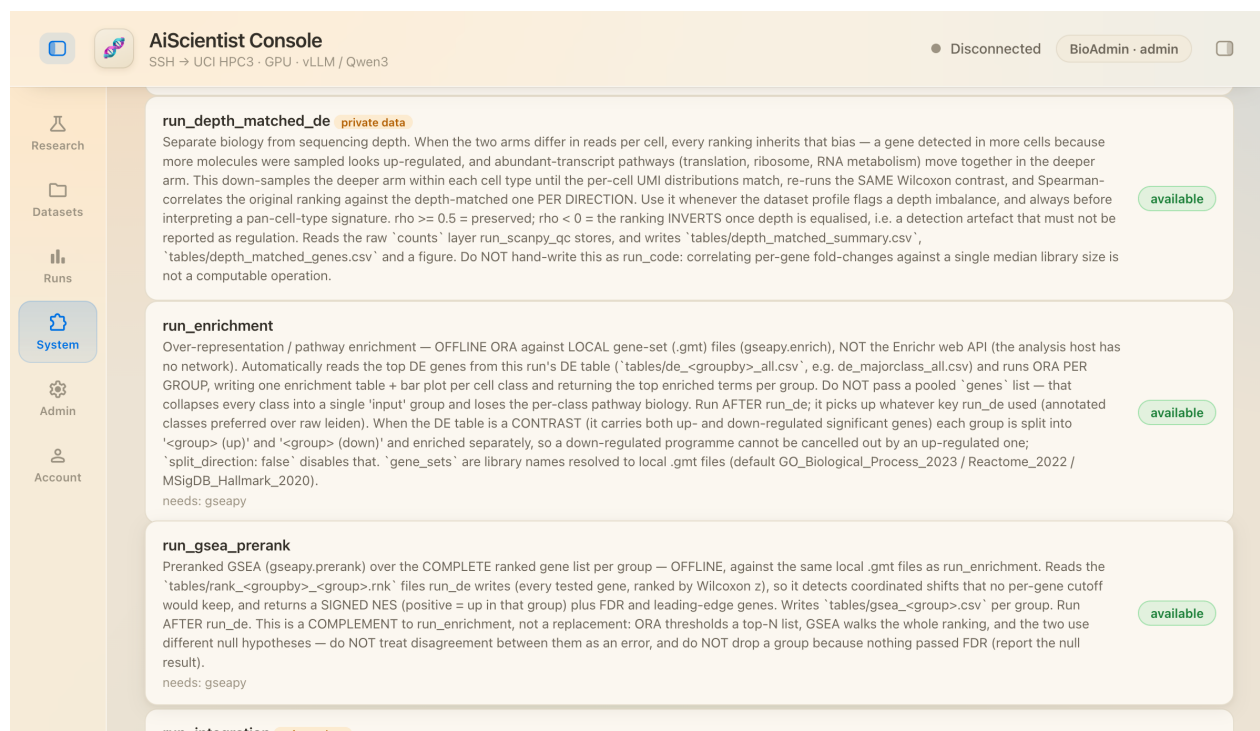


Figure 9: The tool roster on the System page (excerpt)

The single-cell analysis line

Tool	What it does
run_scanpy_qc	Real scanpy QC: per-cell metrics, cell and gene filtering, normalisation, log1p, HVG selection
run_doublet_detection	Scrublet doublet scoring, after QC and before clustering
run_integration	Corrects sample, donor and batch effects before clustering
run_clustering	PCA → neighbours → Leiden → UMAP; optional stability-selected resolution
run_marker_annotation	Assigns a cell type per cluster from a marker panel, with lineage-specific discriminators
run_de	Differential expression (Wilcoxon) in marker or contrast form, stratifiable by cell type
run_pseudobulk_de	Condition-level DE aggregated to one profile per sample (pydeseq2 NB Wald)
run_depth_matched_de	Re-runs the same test after quantile-matching UMI depth, Spearman per direction
run_composition	Cell-type proportions and their shift between conditions
run_enrichment	Offline ORA against local .gmt sets, per group and per direction
run_gsea_prerank	Preranked GSEA over each group's complete ranked list, offline
scgpt_annotate	scGPT reference-based per-cell annotation (a separate short-lived GPU batch job)

The variant and phenotype lines

Tool	What it does
annotate_variants	VCF consequence, affected gene, predicted impact, ClinVar significance; production path is offline VEP with a cache on HPC3

Tool	What it does
map_phenotype_to_hpo	Free text in any language to validated HPO terms; the ontology owns the IDs
run_lirical	Phenotype-driven differential diagnosis by calibrated post-test probability
diagnose_disease	An adjudicated differential combining LIRICAL probability with PaperQA2 evidence

The literature line

Tool	What it does
literature_search	Europe PMC keyword retrieval for real citations; queries built from the accepted findings
deep_literature	PaperQA2 retrieval-augmented generation returning a grounded, cited answer, entirely on HPC3

Code and files

Tool	What it does
run_code	The code sandbox: reads the run's dataset and checkpoints, writes new artifacts; an HPC3 Slurm job capped at 64 GB / 8 CPU / 1 hour
list_dir · stat_path · find_files · read_text · disk_usage	Read-only inspection of the HPC3 filesystem
run_shell · install_package · fetch_url	Controlled shell, package installation and URL fetching on HPC3

Meta tools

Tool	What it does
inspect_dataset	Skims any uploaded file and returns its format, assembly, sample IDs
read_tool_source	Reads the real source of the analysis tools
search_skills · read_skill_reference	Find atomic skills by keyword, then read their guidance and code by progressive disclosure
make_schematic	Describe a structure as text; a deterministic renderer draws the diagram
finish	End the step and return the final answer

S3 Technology stack

Tier 1 — the web console

Category	Contents
Runtime	Host systemd unit <code>bioagent.service</code> , bound to the node IP on <code>:8800</code>
Exposure	Selector-less k8s Service → Envoy Gateway
Server	FastAPI + WebSocket (uvicorn), Python ≥ 3.10
Frontend	Vanilla JS, no framework; Cytoscape (DAG view), Mermaid (inline diagrams)
Accounts	SQLAlchemy 2.0 + Alembic; bcrypt; itsdangerous signed session cookies
Database	PostgreSQL 17 (psycopg 3) in production; SQLite for development and CI
Transport	paramiko SSH + Duo; per-session tunnels; chunked resumable upload; a dedicated transfer host
Email	UCI SER SMTP (STARTTLS + AUTH)

Tier 2 — HPC3 compute

Category	Contents
Cluster	UCI HPC3, Slurm
Containers	Singularity 3.11.3
Serving	vLLM, QuantTrio/Qwen3.6-35B-A3B-AWQ, OpenAI-compatible /v1, dynamic port, per-user isolation
Serving flags	awq_marlin quantization; --max-model-len 262144; memory utilisation 0.92; tool parser qwen3_coder; reasoning parser qwen3
GPU request	Partition gpu, account ruic20_lab_gpu, --gres gpu:A100:1, 8 CPU / 32 GB / 2 hours
Analysis	scanpy · anndata · leidenalg · gseapy · pydeseq2 · matplotlib
CPU jobs	Partition standard; sandbox capped at 64 GB / 8 CPU / 1 hour
Variants	Offline VEP with a cache; bcftools 1.21 / samtools / tabix / bedtools; pysam / cyvcf2; CADD, REVEL, AlphaMissense, OpenSpliceAI
Phenotype	LIRICAL 2.4.1 (with Exomiser hg19/hg38 data)
Literature	Europe PMC; PaperQA2 (local model and local embeddings)
Annotation	scGPT (a separate short-lived GPU batch job)
Reporting	pandoc → XeLaTeX (PDF) and DOCX; graphviz; optional vision-model review
Storage	/dfs3b/ruic20_lab; shared scratch swept every 3 days

Dependency groups. `pyproject.toml` splits dependencies into extras so the core and CI stay light: the core is `h5py` alone; gateway is `paramiko`, `fastapi` and `uvicorn`; auth is `sqlalchemy`, `bcrypt`, `itsdangerous`, `psycopy` and `alembic`; analysis is `scanpy`, `pydeseq2`, `anndata`, `gseapy`, `matplotlib`, `leidenalg` and `pandas`; literature is `paper-qa[local]`; and `langgraph` is a separate extra, imported lazily so the default planners are unaffected when it is absent. System packages: `pandoc`, `texlive-xetex`, `graphviz`.

Quality gates. 1,611 offline tests; ruff static analysis (E4, E7, E9, F); GitHub Actions with four required checks on main.

S4 Glossary

Term	Meaning
PI	The Principal Investigator role; turns a question into an ordered agenda
Scientist	The executing role; calls tools step by step
Critic	The reviewing role; returns accept or revise with a score
agenda	The ordered list of steps the PI produces
Plan mode	The human review pause before the agenda runs
run	One complete execution, with a unique id and its own artifact directory
bundle	The downloadable results package for a run
HarnessTool	The uniform tool wrapper (name, description, JSON schema, executor)
preset pipeline	A packaged analysis pipeline
skill	A rewritable atomic skill
eyeserver	The lab server hosting the web tier
HPC3	UCI's research cluster

S5 Key environment variables

The code defines 207 `BIOAGENT_*` variables; all also accept the `AISCIENTIST_*` prefix.

Variable	Default	Notes
<code>BIOAGENT_HPC_HOST</code>	<code>hpc3.rcic.uci.edu</code>	SSH target
<code>BIOAGENT_HPC_TRANSFER_HOST</code>	<code>access-hpc3.rcic.uci.edu</code>	File-transfer host
<code>BIOAGENT_SLURM_PARTITION</code>	<code>gpu</code>	GPU partition
<code>BIOAGENT_SLURM_ACCOUNT</code>	<code>ruic20_lab_gpu</code>	Billing account

Variable	Default	Notes
BIOAGENT_SLURM_GRES	gpu:A100:1	GPU request
BIOAGENT_GPU_CANDIDATES	empty	Race several (partition, gres, account) candidates
BIOAGENT_LAB_STORAGE	/dfs3b/ruic20_lab	Research data root
BIOAGENT_HPC_SHARED_ROOT	.../software/AiScientist	Containers, model cache, scratch
BIOAGENT_VLLM_MODEL	QuantTrio/Qwen3.6-35B-A3B-AWQ	Session model
BIOAGENT_VLLM_MAX_MODEL_LEN	262144	Context window
BIOAGENT_VLLM_QUANTIZATION	awq_marlin	Empty lets vLLM auto-detect
BIOAGENT_LAB_LLM_BASE_URL etc.	empty	Role split: point PI/Critic/writer at another model
BIOAGENT_SECRET_KEY	no default	Signs session cookies; must be strong and unique
BIOAGENT_DATABASE_URL	—	Production: postgresql+psycpg://...
BIOAGENT_AGENT_MEMORY	0	Per-agent evolving memory (on in production)
BIOAGENT_PLANNER	linear	Set to dag for the DAG planner
BIOAGENT_MAX_CONCURRENCY	1	Parallelism across independent branches

HPC3 offload switches: BIOAGENT_UPLOADS_ON_HPC, BIOAGENT_ANALYSIS_ON_HPC, BIOAGENT_REPORT_ON_HPC, BIOAGENT_RUN_CODE_ON_HPC, BIOAGENT_VARIANT_ON_HPC, BIOAGENT_PHENOTYPE_ON_HPC, BIOAGENT_PAPERQA_ON_HPC.

S6 Path conventions

eyeserver (production)

/data/BioAgent/app/	application root
/data/BioAgent/app/.env	configuration and secrets
/data/runs/	run-artifact persistent volume

HPC3

/dfs3b/ruic20_lab/	lab storage root
/dfs3b/ruic20_lab/<UCInetID>/uploads/	user uploads
/dfs3b/ruic20_lab/software/AiScientist/	shared root
containers/	vllm.sif - analysis.sif - report.sif - vep.sif - scgpt.sif - vlreview.sif
hf/	HuggingFace cache (on shared DFS, not \$HOME)
scgpt_model/	scGPT weights
Temp/	per-run process files, swept every 3 days

S7 Repository layout

src/bioagent/

agents/	the research lab, DAG planner, agent memory, tool registry, code sandbox, provenance
gateway/	web console: SSH+Duo, Slurm vLLM, accounts, chat history, uploads, report assembly
tools/	real analysis, literature, report, scGPT annotation, visual review, schematics, variants, phenotype
lab/	research-lab support
providers/	OpenAI-compatible LLM client
integrations/	the DataBoundaryGuard safety layer
hpc/	Slurm boundaries
core/	shared config, brand env aliases

frontend/console/	the browser UI (served from disk per request)
deploy/	systemd unit, k8s/Envoy, nginx, HPC3 container definitions, redeploy kit
skills/	15 rewritable atomic skills
preset_pipelines/	7 preset pipelines
experiments/	5 measurements and validations
docs/	design documents, ADRs, specifications, workflows
handoff/	per-workstream handoff documents
tests/	126 files, 25,981 lines, 1,611 cases
reports/	dated progress reports and this manuscript

S8 Supplementary figures

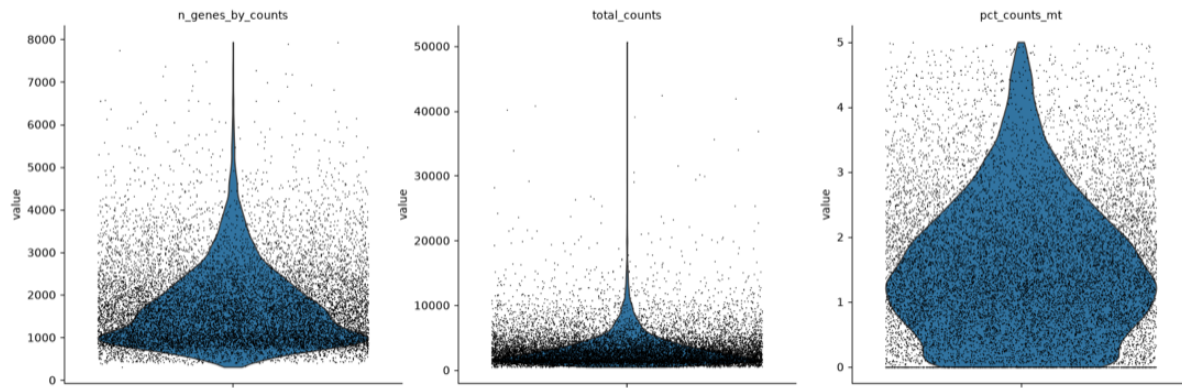


Figure 10: QC violins: per-cell gene counts, UMI counts and mitochondrial fraction

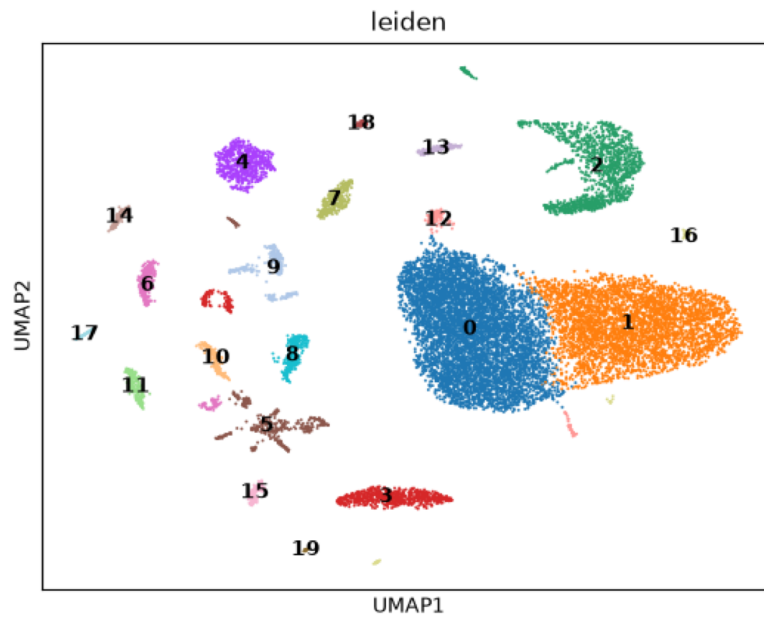


Figure 11: UMAP: the 20 Leiden clusters at the stability-selected resolution

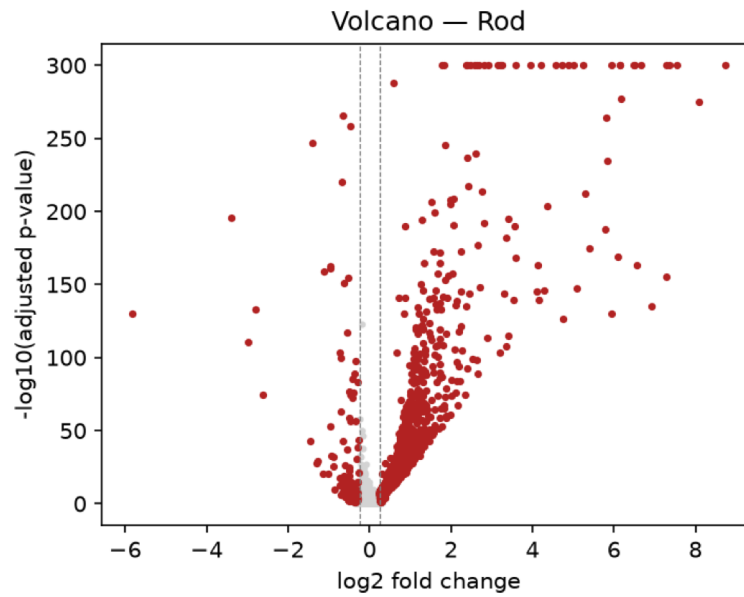


Figure 12: Volcano (Rod): log2 fold change against adjusted p-value

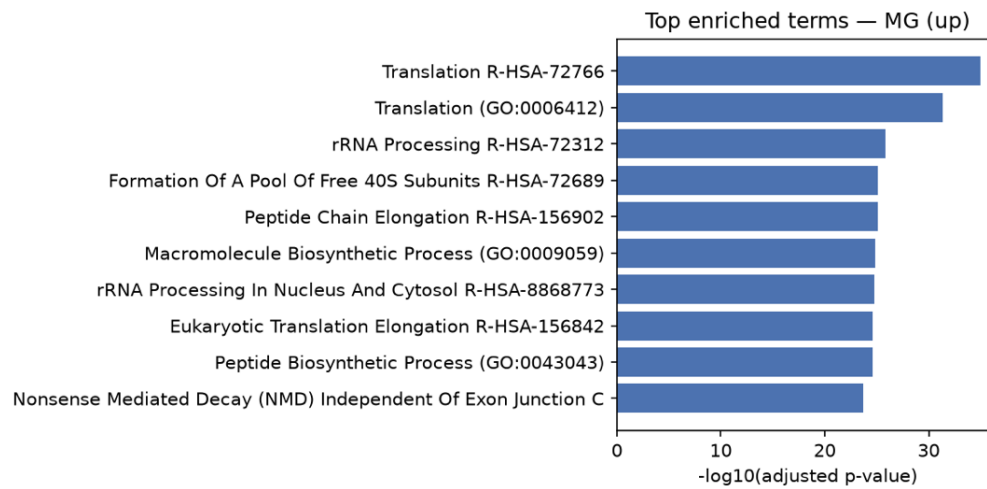


Figure 13: Pathway enrichment (MG, up direction): offline ORA against local gene sets

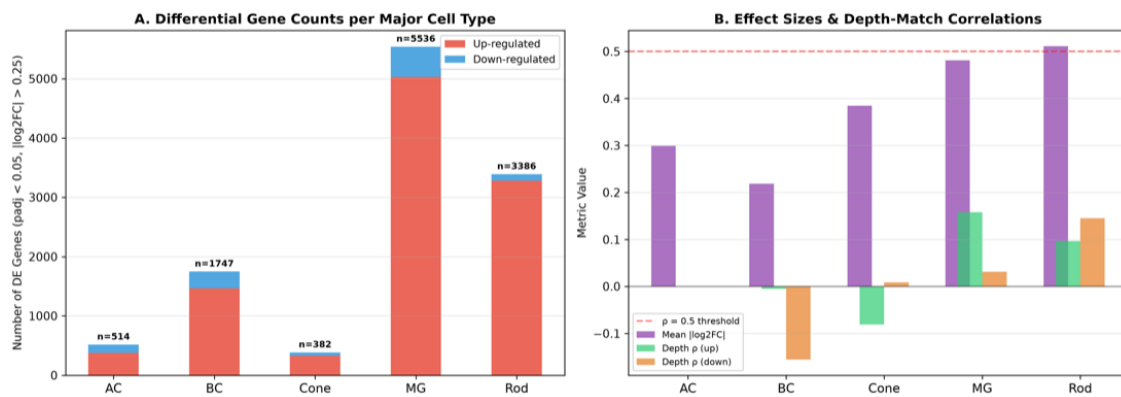


Figure 14: Differential-expression summary: significant gene counts per cell type, split up and down

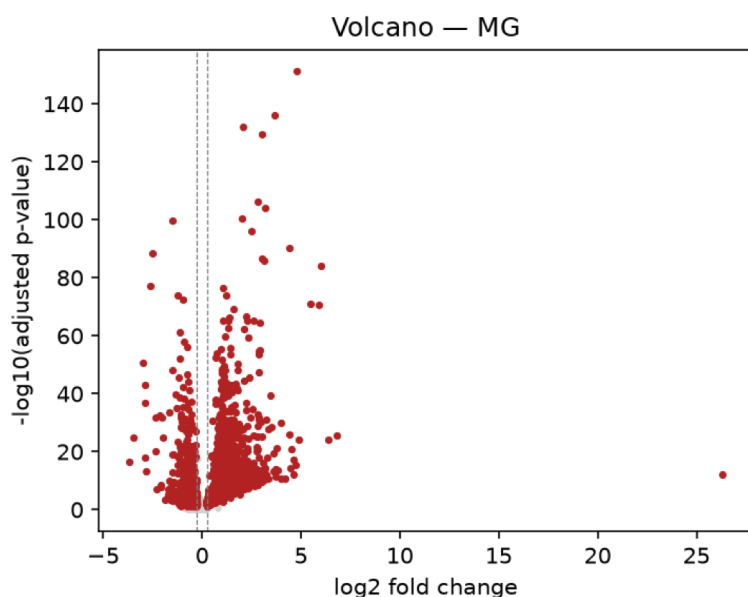


Figure 15: Volcano (MG): the cell class with the largest effect sizes in this run, reaching an abs log2FC of 26.32

S9 Sources

Claim	Source
Source, test and commit counts	Repository statistics, 2026-09-05
Architecture, deployment, dependency groups	README.md, pyproject.toml, deploy/README.md
The role loop, the guard, the plan rules	src/bioagent/agents/, handoff/yijun/HANDOFF.md
Tool descriptions	Extracted from the tool declarations in src/bioagent/**/*.py
Interface screenshots	Captured from the production interface, 2026-09-05
Sample run c135ae589d96	The results bundle: report/, figures/, tables/
The nine-model evaluation	experiments/plan_vs_exec_ab/ (2026-08-19, two rounds)
Plan-revision validation	experiments/plan_revision_ab/
Depth-matching validation	experiments/depth_matched_validation/, result-2026-09-02.log (post-fix)
Meeting evidence distribution	experiments/meeting_asymmetry/
Protocol-format A/B	experiments/protocol_format/
KV cache and context measurements	ctxprobe jobs, 2026-08-02

Claim	Source
Node and partition characteristics	handoff/yijun/HANDOFF.md, 2026-08-10
NVFP4 weight sizes and serving stack	deploy/dsv4/ and subsequent validation records
MiniMax-M2.7 / M3 serving measurements	Deployment trials on the 4×96 GB node (vLLM 0.22.1 / 0.27.1 / nightly), recorded 2026-09
Blackwell device memory	NVIDIA product pages and vendor-published specifications (RTX PRO 6000 / 5000 / 4500: 96 / 48 / 32 GB GDDR7; B200: 180 GB HBM3e; B300: 288 GB HBM3e)
Default configuration values	src/bioagent/gateway/settings.py